

Aula 7 | Funções e Assinaturas



Objetivos da aula de hoje

Ao final, você deve conseguir:

- Resolver o problema: transformar o jogo de dados em uma função reutilizável que pode ser chamada por qualquer parte do sistema.
- Entender e aplicar: declaração de funções, parâmetros com padrão, retorno, assinatura, arrow functions e composição (pipe) de funções.

Problema gerador

Transformar o jogo de dados em uma funcionalidade reaproveitável

Pergunta guia

Como transformar o simulador de dados em uma função que qualquer parte do sistema pode chamar – passando parâmetros, recebendo resultados e podendo ser combinada com outras funções?

- Estudo de caso 1: Refatorar o jogo de dados como função reutilizável
- Expositiva: Funções, assinaturas, parâmetros e retorno
- Expositiva: Arrow functions e composição de funções
- Prático: Exercícios de fixação (funções)



Conceitos

Funções e Assinaturas

- `function declaration` – hoisting, reutilização, nomeada
- `function expression` – atribuída a uma variável
- Parâmetros e valores padrão (`param = default`)
- `return` – o que a função entrega ao chamador
- Assinatura – nome + parâmetros + tipo de retorno
- Arrow function – sintaxe concisa, retorno implícito
- Composição – saída de uma função vira entrada de outra (pipe)

Declaração de Funções em JavaScript

FUNÇÃO:

function declaration sofre hoisting: pode ser chamada antes de aparecer no código. function expression não: a variável só existe após a linha. Prefira declaração para funções principais e expressão para callbacks e valores.

```
// 1) Function declaration (sofre hoisting)
```

```
function saudar(nome) {  
    return 'Olá, ' + nome + '!';  
}  
console.log(saudar('Ana')); // Olá, Ana!
```

```
// 2) Function expression (NÃO sofre hoisting)
```

```
const dobrar = function(n) {  
    return n * 2;  
};
```

```
console.log(dobrar(5)); // 10
```

```
// Pode chamar declaration ANTES de declarar:
```



Parâmetros, Retorno e Assinatura

Parâmetro com padrão (param = valor): se o chamador não passar o argumento, usa o padrão. `return` encerra a função e entrega o valor ao chamador. Uma função deve fazer UMA coisa bem – princípio da responsabilidade única.

A assinatura de uma função define seu contrato: nome, entradas e saída.

Parâmetros com padrão evitam erros quando o chamador omite um argumento.

```
// Assinatura: rolarDado(lados, vezes)
function rolarDado(lados = 6, vezes = 1) {
  const resultados = [];
  for (let i = 0; i < vezes; i++) {
    resultados.push(Math.floor(Math.random() * lados) + 1);
  }
  return resultados;
}

// Chamadas válidas:
rolarDado();           // [3]  (6 lados, 1 vez)

rolarDado(20);         // [17] (20 lados, 1 vez)

rolarDado(6, 3);       // [4, 2, 6] (3 vezes)

// return sem valor = undefined
// Função sem return também = undefined
// Sempre documente o que retorna!
```



Estudo de Caso 1: Dados como Função

Refatorar = melhorar a estrutura sem mudar o comportamento. rolarDado faz UMA coisa: sorteia 1 valor. simularDados
orquestra: chama rolarDado N vezes e agrega. reduce() acumula valores – preview do próximo tema.

```
// Antes (Aula 6): código solto, não reutilizável
// Agora: funções com assinatura clara

function rolarDado(lados = 6) {

    return Math.floor(Math.random() * lados) + 1;

}

function simularDados(n = 1, lados = 6) {
    const rolagens = [];

    for (let i = 0; i < n; i++) rolagens.push(rolarDado(lados));
    const soma = rolagens.reduce((acc, v) => acc + v, 0);
    return { rolagens, soma, media: (soma / n).toFixed(1) };
}
```



Arrow Functions

```
// const f = function(n){ return n*2; }; // expression
```

Sintaxe concisa para funções simples – muito usada em callbacks.

```
// Sintaxe: (parâmetros) ⇒ expressão  
// Retorno implícito quando só tem 1 expressão:  
const dobrar = n ⇒ n * 2;
```

```
const somar = (a, b) ⇒ a + b;
```

```
const saudar = nome ⇒ 'Olá, ' + nome;
```

```
// Com bloco {}: retorno explícito obrigatório  
const rolarDado = (lados = 6) ⇒ {  
    return Math.floor(Math.random() * lados) + 1;  
};
```

```
// Arrow functions não têm próprio `this`  
// Use function clássica quando precisar de `this`
```

```
// Comparando os 3 estilos:
```

```
// function f(n) { return n*2; } // declaration
```



Composição e Encadeamento de Funções

Composição: saída de uma função vira entrada de outra.

- Cada função faz UMA coisa – fácil de testar e reutilizar
- Pipeline: `f3(f2(f1(entrada)))` – encadeia transformações
- Mais legível que uma função gigante com tudo dentro

Exemplo – pipeline de notas:

```
const calcMedia = notas => notas.reduce((s,n)=>s+n,0)/notas.length;  
const arredondar = (v, d=1) => Number(v.toFixed(d));
```

```
const classificar = m => m >= 6 ? 'Aprovado' : 'Reprovado';
```

```
// Composição manual (pipe):  
const notas = [7, 8, 6, 9];
```

```
const media = arredondar(calcMedia(notas));  
const resultado = classificar(media);  
console.log(`Média: ${media} – ${resultado}`);
```



Exercícios de fixação

Grupo 1: “Calculadora de dados”

Refatore o simulador da Aula 6 como funções reutilizáveis.

EX: // use as funções `rolarDado` e `simularDados` do EC1

1. Escreva ``rolarDado(lados = 6)`` que retorna 1 valor sorteado.
2. Escreva ``simularDados(n, lados)`` que retorna ``{ rolagens, soma, media }``.
3. Reescreva ``simularDados`` como arrow function.
4. Escreva ``maiorRolagem(rolagens)`` que retorna o valor máximo.
5. Escreva ``formatarResultado(obj)`` que retorna uma string formatada para exibir.
6. Componha: ``console.log(formatarResultado(simularDados(5, 20)))``.
7. ****Desafio:**** escreva ``pipe(...fns)`` que encadeia funções automaticamente.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!



Exercícios de fixação

Grupo 2: “Pipeline de texto”

Construa um pipeline de transformações sobre strings.

ex:

```
const frase = ' Olá Mundo! como vai você? ';
```

1. Escreva ``limpar(s)`` que remove espaços extras (`trim`).
2. Escreva ``capitalizar(s)`` que coloca a primeira letra de cada palavra em maiúscula.
3. Escreva ``removerExclamacao(s)`` que remove todos os ``!``.
4. Componha as 3 funções: ``removerExclamacao(capitalizar(limpar(frase)))``.
5. Reescreva cada função como arrow function de 1 linha.
6. Escreva ``contarPalavras(s)`` que retorna o número de palavras.
7. ****Desafio:**** crie um array de funções e aplique com ``reduce``.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!



Exercícios de fixação

Grupo 3: “Mini sistema de notas”

Use funções para processar um array de alunos.

ex:

```
const turma = [  
  { nome: 'Ana', notas: [8, 7, 9] },  
  { nome: 'Bruno', notas: [4, 5, 6] },  
  { nome: 'Carla', notas: [9, 10, 8] },  
];
```

1. Escreva `calcMedia(notas)` que recebe array e retorna a média.
2. Escreva `classificar(media)` que retorna “Aprovado” ou “Reprovado”.
3. Escreva `gerarBoletim(aluno)` que usa `calcMedia` + `classificar` e retorna objeto.
4. Componha: aplique `gerarBoletim` a cada aluno com um `for-of`.
5. Escreva `melhorAluno(turma)` que retorna o aluno com maior média.
6. Reescreva `calcMedia` como arrow function de 1 linha.
7. ****Desafio:**** escreva `gerarRelatorio(turma)` que retorna string com todos os boletins.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!

Correção

```
const rolarDado = (lados = 6) =>

  Math.floor(Math.random() * lados) + 1;

function simularDados(n = 1, lados = 6) {
  const rolagens = [];
  for (let i = 0; i < n; i++)
    rolagens.push(rolarDado(lados));
  const soma = rolagens.reduce((s, v) => s + v, 0);
  return { rolagens, soma, media: (soma/n).toFixed(1) };
}

const maiorRolagem = r => Math.max(...r);

const formatarResultado = ({ rolagens, soma, media }) =>

  `Rolagens: ${rolagens.join(', ')} | Soma: ${soma} | Média: ${media}`;

// Composição:
console.log(formatarResultado(simularDados(5, 20)));
console.log('Maior:', maiorRolagem(simularDados(5).rolagens));
```



Correção

```
const frase = ' Olá Mundo! como vai você? ';  
  
const limpar = s ⇒ s.trim();  
  
const capitalizar = s ⇒  
  s.split(' ')  
    .map(p ⇒ p.charAt(0).toUpperCase() + p.slice(1))  
    .join(' ');  
  
const removerExclamacao = s ⇒ s.replace(/!/g, '');  
  
const contarPalavras = s ⇒ s.trim().split(/\s+/).length;  
  
// Pipeline composto:  
  
const resultado = removerExclamacao(capitalizar(limpar(frase)));  
console.log(resultado); // 'Olá Mundo Como Vai Você?'  
console.log('Palavras:', contarPalavras(resultado));  
  
// Desafio: pipe com reduce  
  
const pipe = (...fns) ⇒ x ⇒ fns.reduce((v, f) ⇒ f(v), x);  
const processar = pipe(limpar, capitalizar, removerExclamacao);
```



Correção

```
const calcMedia = notas =>
  notas.reduce((s, n) => s + n, 0) / notas.length;

const classificar = m => m ≥ 6 ? 'Aprovado' : 'Reprovado';

function gerarBoletim(aluno) {
  const media = Number(calcMedia(aluno.notas).toFixed(1));

  return { nome: aluno.nome, media, status: classificar(media) };
}

const turma = [

  { nome: 'Ana', notas: [8,7,9] },
  { nome: 'Bruno', notas: [4,5,6] },

  { nome: 'Carla', notas: [9,10,8] },

];
const boletins = [];
for (const a of turma) boletins.push(gerarBoletim(a));
console.log(boletins);

const melhorAluno = t => t.reduce((top, a) =>
  calcMedia(a.notas) > calcMedia(top.notas) ? a : top);
```



Pesquisa Individual

Funções, Arrow Functions e Callbacks

Leituras recomendadas:

- MDN Functions: developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Functions
- MDN Arrow:
developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Functions/Arrow_functions

Perguntas de pesquisa (10–15 min)

- 1) O que é hoisting? Por que function declarations sofrem hoisting e expressions não? (Dica: o JS "move" declarações para o topo do escopo antes de executar)
- 2) Arrow functions não têm `this` próprio. Quando isso importa?
- 3) O que é uma função pura? Por que funções puras são fáceis de testar?
- 4) O que é o princípio da responsabilidade única? Dê 1 exemplo prático.
- 5) Bônus: pesquise `Array.prototype.reduce` – como funciona o acumulador?



Próxima aula

Aula 8

Métodos funcionais de arrays: map, filter e reduce – transformar, filtrar e acumular dados com funções.

Para chegar bem:

- Traga dúvidas sobre funções e os exercícios
- Complete os 3 exercícios de fixação
- Pesquise a diferença entre função pura e impura



Obrigado